Discrete Diffusion Policies are Natural Asynchronous Executors

Anonymous Author(s)
Affiliation
Address
email

Abstract: Deploying high-latency generative policies in real-time robot control requires asynchronous execution, where the next action chunk is inferred while the current one is being executed. Real-time chunking (RTC) achieves this for continuous diffusion and flow-matching policies via gradient-based inpainting, but suffers from five key limitations: inability to leverage pre-training, need for inpainting-specific fine-tuning, reliance on heuristic guidance weights, extra inference-time computation from backpropagation, and a distribution mismatch between inpainting conditions at inference versus training. We observe that discrete diffusion policies—which generate actions by iteratively unmasking tokens—are *natural* asynchronous executors: inpainting is their native operation, eliminating the first four limitations without any architectural or algorithmic modification. To address the remaining distribution mismatch, we propose a fine-tuning recipe that exposes the model to its own predictions as conditioning context during training. Experiments on simulated dynamic control benchmarks demonstrate that DiscreteRTC outperforms continuous RTC and other asynchronous baselines in both success rate and throughput, while being simpler and computationally cheaper.

Keywords: Discrete Diffusion, Real-Time Control, Asynchronous Execution

Main Demo PlaceHolder

Main Demo PlaceHolder

Main Demo PlaceHolder

Main Demo PlaceHolder

Main Demo PlaceHolder

Main Demo PlaceHolder

Main Demo PlaceHolder

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

1 Introduction

Physical AI systems, from dexterous manipulators to mobile robots, must act in real time—the world does not pause while a policy deliberates. However, the most capable modern policies, including billion-parameter vision-language-action models (VLAs) [**? ? ?**], require hundreds of milliseconds per inference call, far exceeding the control periods of typical robotic systems. This latency gap is unlikely to close: as robot datasets grow, so will the models that learn from them [**?**], and remote inference adds further overhead.

Asynchronous execution offers a principled solution: a policy can *think about its next actions while executing the current ones*, effectively hiding inference latency behind action execution. Action chunking [**? ?**], where a policy outputs a sequence of future actions per inference call, provides the temporal structure needed for asynchronous execution. Yet naively switching between consecutively generated action chunks causes inter-chunk discontinuity: adjacent chunks may correspond to different behavioral modes, producing jerky, out-of-distribution motions at chunk boundaries [**?**].

Real-time chunking (RTC) [**?**] addresses this by posing asynchronous execution as an *inpainting* problem: the new action chunk is generated conditioned on the prefix of already-committed actions from the previous chunk, ensuring smooth transitions. For continuous diffusion and flow-matching policies, RTC implements inpainting via gradient-based guidance (ΠGDM) [1**?**], adding a correction term at each denoising step that steers the generation toward consistency with the frozen prefix. While effective, this approach inherits five concrete limitations:

No pre-training benefit. The continuous policy is not pre-trained for inpainting; the guidance mechanism is imposed at inference and is foreign to the learned distribution. **Inpainting-specific fine-tuning required.** Achieving strong inpainting performance requires additional fine-tuning with the guidance mechanism. **Heuristic guidance weights.** The soft-masking schedule and gradient clipping threshold are hand-designed hyperparameters that require task-specific tuning. **Extra inference cost.** Gradient-based guidance requires backpropagation through the denoising network at every denoising step, roughly doubling compute.

Our key observation is that *discrete diffusion policies*—which generate action sequences by iteratively unmasking tokens from a fully masked state—perform inpainting *by construction*. In a discrete diffusion model, conditioning on a subset of known tokens while generating the rest is not an auxiliary mechanism bolted on at inference; it is the *native operation* of the model, identical to its pre-training and fine-tuning objective. This structural property resolves limitations (a) through (d) simultaneously: pre-training directly teaches inpainting (a), no special fine-tuning is needed (b), there are no guidance weights to tune (c), and no gradient computation is required at inference (d).

We propose DiscreteRTC, which replaces the gradient-based inpainting in RTC with the native masked generation of discrete diffusion policies. Experiments on dynamic simulated benchmarks demonstrate that DiscreteRTC achieves higher success rates and throughput than continuous RTC and other asynchronous baselines, while being simpler to implement and computationally cheaper.

In summary, our contributions are:

1. We provide a systematic analysis of the fundamental limitations of the currently popular RTC framework with a Flow-Matching action head.
2. We show that these limitations can be naturally resolved by discrete diffusion policies. In this sense, **Discrete Diffusion Policies are Natural Asynchronous Executors.**
3. Through simulation and real-world experiments, we demonstrate that the resulting method, **DiscreteRTC**, (a) achieves higher task success rates and throughputs under varying delays, (b) outperforms training-time methods that require fine-tuning, (c) uses less inference time, (d) and can be integrated with advanced inference-time methods such as VLASH [2].

2 Background and Preliminaries

2.1 Real-time Chunking

Action Chunking Flow Policy. We consider an action chunking policy π that receives an observation $\mathbf{o}_t$ and outputs a chunk of H future actions $\mathbf{A}_t = (a_t, a_{t+1}, \ldots, a_{t+H-1})$. State-of-the-art VLAs [3, 4] generate action chunks using conditional flow matching [5]: noise $\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$ is sampled and integrated through the learned velocity field v_π from $\tau = 0$ to 1:

$$\mathbf{A}_t^{\tau+\frac{1}{n}} = \mathbf{A}_t^\tau + \frac{1}{n} v_\pi(\mathbf{A}_t^\tau, \mathbf{o}_t, \tau), \tag{1}$$

where $\tau \in [0, 1)$ is the flow timestep and n is the number of denoising steps. During training, all actions within a chunk are corrupted at the same noise level τ and denoised together.

Real-Time Chunking. Running modern VLAs with action generation time δ smaller than the controller period Δt (typically 20$\sim$50 ms) is nearly impossible [6]: even a 7B OpenVLA model optimized for speed [7, 8] requires $\delta = 321$ ms on a server-grade A100 GPU. The robot must therefore act while the policy is still computing, *i.e.*, asynchronous execution. Naively switching to a new action chunk as soon as it is ready, however, leads to inter-chunk discontinuity: adjacent chunks may correspond to different behavioral modes, producing jerky motions at chunk boundaries.

Real-time chunking (RTC) [6] addresses this discontinuity by posing asynchronous execution as an *inpainting* problem. If inference takes d steps to complete, the first d actions of the new chunk are "frozen" to the values from the previous chunk before the new chunk becomes available. The remaining actions must be generated to be consistent with this frozen prefix. For flow-matching policies, RTC implements this inpainting via ΠGDM [1], adding a gradient-based correction at each denoising step that steers the generation toward the target prefix $\mathbf{Y}$:

$$\mathbf{g} = \left(\mathbf{Y} - \hat{f}_{\mathbf{A}^1}(\mathbf{A}^\tau)\right)^\top \operatorname{diag}(\mathbf{W}) \cdot \frac{\partial \hat{f}_{\mathbf{A}^1}}{\partial \mathbf{A}'}\bigg|_{\mathbf{A}'=\mathbf{A}^\tau}, \tag{2}$$

where $\hat{f}_{\mathbf{A}^1}$ denotes the one-step denoising function and $\mathbf{W}$ is a vector of soft-masking weights. The weights $\mathbf{W}$ assign 1 to the first d frozen actions, exponentially decay for the intermediate overlapping region, and 0 for the final s actions. During inference, the robot starts an inference job every s steps and execute the next s actions from the most recent inference results.

2.2 Discrete Diffusion Policies

Discrete diffusion policies first tokenize continuous actions into discrete tokens via a learned or fixed vocabulary (*e.g.*, VQ-VAE [9], FAST [10] or k-bin quantization). With a slight abuse of notation, we reuse $\mathbf{A}_t$ to denote the resulting action token sequence. The forward process replaces tokens with a special [MASK] token with a masking schedule, while the reverse process iteratively executes unmasking operations conditioned on the current fully or partially masked token sequence.

A discrete diffusion policy is trained for one-step unmasking operation. It consists of two components: p_π, which predicts the probability distribution over all token positions, and f_π, which samples and select the tokens should be unmasked at the current step from the distribution. At each step k, the policy first predicts token identities via p_π and then selects which tokens to unmask via f_π:

$$\mathbf{A}_t^{k+1} = \pi(\mathbf{A}_t^k, \mathbf{o}_t) = f_\pi\big(p_\pi(\mathbf{A}_t^k, \mathbf{o}_t)\big), \tag{3}$$

where $\mathbf{A}_t^k$ is the partially unmasked token sequence at step k and $\mathbf{o}_t$ is the observation. During training, the discrete diffusion policy learns to predict the fully unmasked ground-truth action token sequence given the randomly masked token sequence.

3 Flow-Matching is not suitable for RTC

In this section, we analyze the limitations of applying flow-matching policies to real-time chunking, as illustrated in Figure 1, establishing the motivation for our discrete diffusion-based approach.

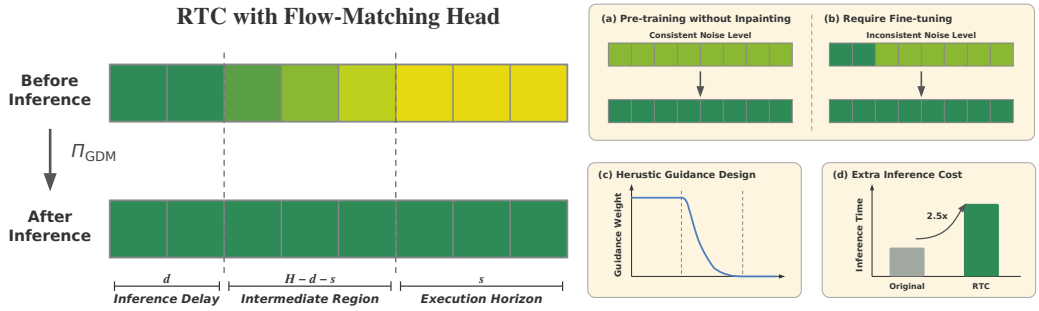

Figure 1: RTC with a flow-matching head. The flow-matching head is ill-suited for RTC because (a) during pre-training, the base policy is not trained on inpainting tasks; (b) to acquire this capability, a specially designed fine-tuning stage is required; (c) at inference time, RTC relies on heuristic guidance weight design; and (d) incurs extra computation for action chunk generation.

3.1 Pre-training without Inpainting

Pre-training of flow-matching policies learns to generate action chunks conditioned on observations and a noised action chunk. During training, all actions within a chunk are corrupted at the **consistent** noise level τ and are denoised together under a single computational budget. However, during RTC, the generation process starts from an action chunk with **inconsistent** noise levels across positions. Although the ΠGDM guidance provides an external correction at inference time, the model has never encountered such corrections—nor the mixed-noise chunk pattern—during pre-training. As a result, scaling up pre-training data or model capacity does not directly improve inpainting quality.

Therefore, to improve the inpainting capability of the base model, the mixed-noise chunk pattern must be explicitly introduced during training along with a corresponding inpainting objective. This requires fine-tuning the model with an explicit inpainting loss, adding training complexity and risking interference with the base generation quality. This represents a non-trivial engineering and computational burden, particularly for large VLAs that are already expensive to fine-tune. Moreover, the fine-tuning recipe itself must be carefully designed: for models pre-trained at large scale, a correspondingly large-scale fine-tuning effort may be necessary to reliably acquire the inpainting capability without degrading other abilities.

3.2 Inference with Heuristic Corrections

As mentioned in the Sec. 2.1, the forward of RTC with flow-matching is presented as follows:

$$\mathbf{A}_t^{\tau+\frac{1}{n}} = \mathbf{A}_t^{\tau} + \frac{1}{n}\left(v_\pi(\mathbf{A}_t^{\tau}, \mathbf{o}_t, \tau) + \min\left(\beta, \frac{1-\tau}{\tau \cdot r_\tau^2}\right)\mathbf{g}\right). \tag{4}$$

The soft-masking weights W in Equation 2 follow a hand-designed exponential decay schedule. This schedule is fixed across different inference situations—varying delays, execution horizons, and tasks—and is validated only through empirical studies. The gradient clipping threshold β, introduced to prevent instability with few denoising steps, is similarly sensitive: its optimal value depends on the task, the model architecture, and the number of denoising steps. Together, these hyperparameters create a tuning burden that is absent in synchronous execution and must be revisited whenever the model, task, or hardware configuration changes.

Moreover, at each denoising step, computing the guidance term requires a vector-Jacobian product (VJP). This roughly doubles the per-step computation, directly counteracting the latency reduction that asynchronous execution is meant to provide. There is an ironic tension: the mechanism designed to enable smooth asynchronous execution itself increases the inference delay d, shrinking the time window available for generating fresh actions.

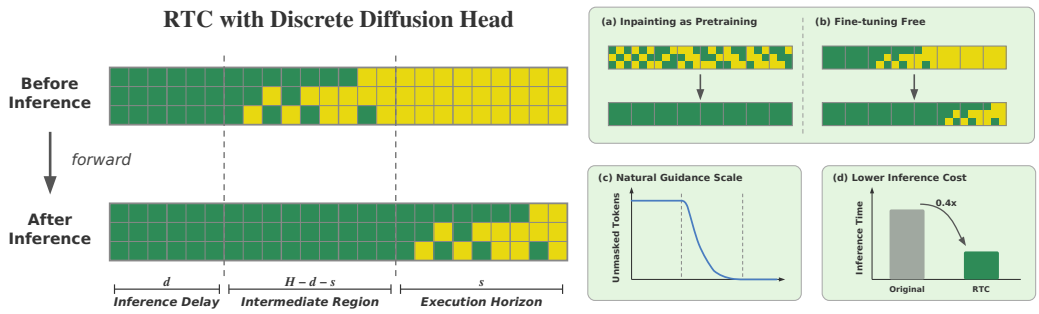

Figure 2: RTC with a discrete diffusion head. The discrete diffusion head is naturally suited for RTC because (a) during pre-training, the base policy is already trained on inpainting tasks; (b) consequently, no inpainting-specific fine-tuning is required; (c) at inference time, early stopping from the previous inference leaves a natural and adaptive guidance signal; and (d) fewer tokens need to be unmasked per step, reducing the computational cost of each forward pass.

4 Discrete Diffusion Policies are Natural Asynchronous Executors

In this section, we show that discrete diffusion policies naturally resolve the limitations of flow-matching-based RTC identified in Section 3, as illustrated in Figure 2.

4.1 Pre-Training with Inpainting

In discrete diffusion, inpainting is structurally identical to standard masked generation: given a partially masked token sequence, the model predicts and unmasks the missing tokens. No architectural modification, no additional loss term, and no inference-time guidance are needed—**the model generates conditioned on unmasked tokens exactly as it was trained**. Therefore, pre-training directly improves the model's ability to inpaint: larger models and more data yield better generation.

Since the inpainting capability is acquired during pre-training, a discrete diffusion policy can be deployed for asynchronous execution immediately, with no inpainting-specific fine-tuning.

4.2 Inference with Natural Unmasking

Since the base policy is pre-trained to solve inpainting, after placing the previous action chunk at the consistent time index, inpainting is achieved by simply forwarding the base policy π:

$$\mathbf{A}_t^{k+1} = \pi(\mathbf{A}_t^k, \mathbf{o}_t). \tag{5}$$

With this formulation, we reconsider the issues of guidance weight and inference cost. Here, we exploit a key property of discrete diffusion: once a token is unmasked, it carries concrete semantic meaning. This fundamentally differs from continuous noisy actions, which become valid only when the noise level reaches zero. Therefore, as indicated in Equation 5, **we can not only start inference from an intermediate state, but also terminate it at an intermediate stage.**

To ensure that executable actions are valid before the next inference, we only need the next s actions in the chunk to be fully unmasked, allowing the unmasking process to stop early. The remaining $H-d-s$ actions stay in a partially unmasked state, which serves as a *natural and adaptive* guidance signal for the next inference, replacing the heuristic soft-masking schedule of continuous RTC.

Moreover, since unmasking starts from an intermediate state and ends at an intermediate stage, keeping the number of tokens unmasked per step constant reduces the total number of unmasking steps to roughly s/H of the original. This means that, instead of *increasing* inference cost as in continuous RTC, our method can actually *reduce* it. Alternatively, we can keep the number of inference steps fixed and reduce the number of tokens unmasked per step, thereby producing finer-grained actions without increasing the overall cost.

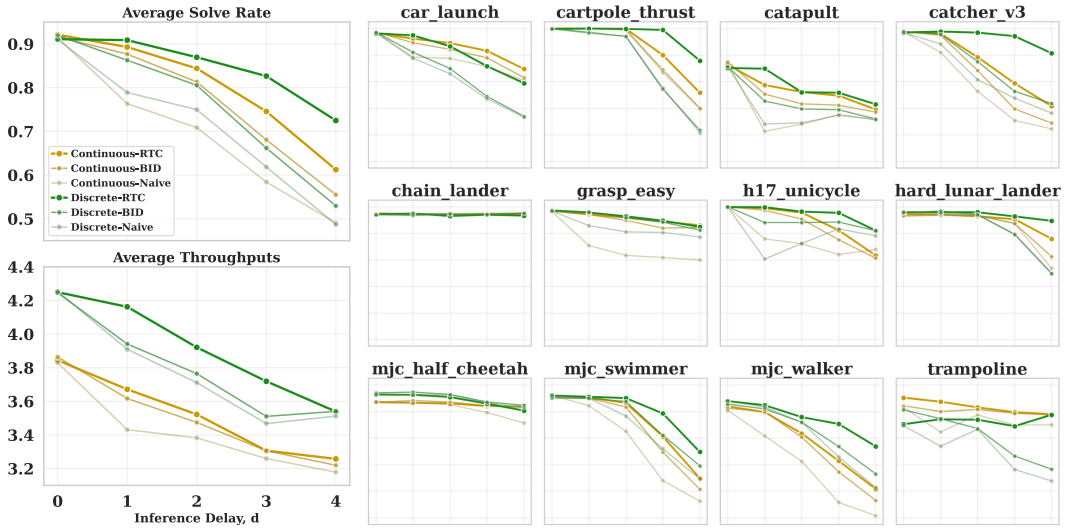

Figure 3: Experimental Results in Kinetix. **Left**: Average solve rate and throughputs across all environments with different inference delays; **Right**: Solve rates for every tasks with different inference delays. The executions horizon follows $s = \max(1, d)$ and each datapoint represents 2048 trials.

5 Experiments

In our experiments, we aim to answer the following questions:

- How does DiscreteRTC performs comparing to existing inference-time and training-time asynchronous execution methods under increasing inference delays?
- How does the natural guidance of DiscreteRTC effect the performance and efficiency?
- How does DiscreteRTC affect the performance and speed of real-world robots?

5.1 Simulated Benchmark

Following RTC, we first evaluate DiscreteRTC in Kinetix [11], a simulated benchmark of dynamic tasks with force-based control, where inference delay necessitates asynchronous execution. All tasks involve dynamic motions such as throwing, catching, and balancing. To simulate imperfect actuation, we add Gaussian noise to the actions, making closed-loop corrections crucial for success.

Setup. We implemented the discrete diffusion policies and datasets upon the official codebase of RTC. To make the comparison fair, we adopt the same training setup (e.g. optimizer, learning rate, epochs, etc.) and the same policy network size except the final layer for logits output in discrete diffusion. We use a trivial 512-bin quantization for the action chunk discretization.

Baselines. We compare against the two archtectures with the following inference-time strategies:

- **Naive Async.** Chunks are switched as soon as the new one is ready, without conditioning on the previous chunk.
- **Bidirectional decoding (BID)** [12]. Uses rejection sampling to maintain cross-chunk continuity. We use a batch size of $N = 16$ with no weak policy as the official implementation.
- **RTC** [6]. Flow-matching policy with ΠGDM inpainting and soft masking, representing the state-of-the-art asynchronous execution method for continuous policies.

Results. We present the main results in Kinetix in Figure 3. Across all inference delays, DiscreteRTC consistently outperform ContinuouRTC and other variants on both solve rates and throughputs, showing the great advantages of the native inpainting capability in discrete diffusion policies. Moreover, as shown in Figure 4, DiscreteRTC requires fewer iterative steps than ContinuousRTC, especially under large delays with execution horizon as discussed in the Sec. 4.2.

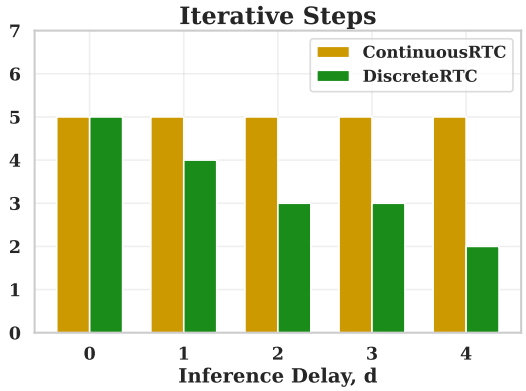

Figure 4: Required iterative steps of different policy architectures in Kinetix with $s = \max(1, d)$.

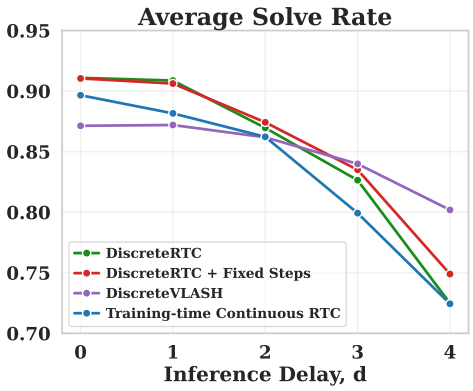

Figure 5: Experimental results of extended baselines in Kinetix with $s = \max(1, d)$.

Figure 5 represents the experimental results of more variants in Kinetix. (1) First, under varying delays, the DiscreteRTC can outperform *Training-time Continuous RTC*, which is fine-tuned with an explicit objective and recipe to improve inpainting, further demonstrating the strong performance of the fine-tuning free DiscreteRTC. (2) Next, by using fixed unmasking steps during inference, the *DiscreteRTC + Fixed* Steps baseline can achieve better performance by generating more fine-grained actions with the same computation budget as discussed in Sec. 4.2. (3) Moreover, *DiscreteVLASH*, which integrates the VLASH [2] can achieve more stable performance across varying delays but at the cost of performance drop with smaller delays. This demonstrates that DiscreteRTC can be seamlessly combined with advanced inference-time methods for further gains.

5.2 Real-World Results

Next, we validate the DiscreteRTC in the real world.

Setup. We develop the .

Tasks.

Baselines.

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Real-Experiments Placeholder

Results. Results Placeholder

Results Placeholder

Results Placeholder

6 Related Work

Efficient VLA. Action chunking [**? ?**] has become the standard in imitation learning for visuomotor control, with expressive generative models—including diffusion [**? ?**], flow matching [**? ?**], variational inference [**?**], and discrete tokenization [**? ?**]—used to model the chunk distribution. Vision-language-action models (VLAs) [**? ? ?**] scale this paradigm to billions of parameters, leveraging pre-trained vision-language backbones and growing robot datasets [**? ?**].

Discrete Diffusion VLA. Discrete tokenization of actions has been explored via VQ-BeT [**?**], byte-pair encoding [**?**], and k-bin discretization. However, the application of discrete *diffusion* (masked generative models) to robot control—and particularly to asynchronous real-time execution—remains largely unexplored. Our work identifies and exploits the structural advantage of discrete diffusion for inpainting-based asynchronous control.

Asynchronous Execution Strategy. RTC [**?**] is the most closely related work, introducing inpainting-based asynchronous execution for continuous flow policies. Bidirectional decoding (BID) [12] uses rejection sampling for cross-chunk consistency but at significantly higher computational cost. Thinking while moving [**?**] considers concurrent control in reinforcement learning but does not address action chunking.

7 Discussion and Future Steps

Conclusions. We presented DiscreteRTC, which exploits the native inpainting capability of discrete diffusion for asynchronous real-time control, eliminating the need for inpainting-specific fine-tuning, heuristic guidance weights, and extra inference cost inherent to flow-matching-based RTC. Simulated and real-world experiments show that DiscreteRTC outperforms continuousRTC and other asynchronous baselines, and can be combined with methods such as VLASH for further gains.

Limitations. Our approach uses naive k-bin quantization, which ignores the temporal structure of action sequences. In addition, the modularized architecture—an autoregressive VLM paired with a separate discrete diffusion action head—prevents the backbone from participating in iterative unmasking, limiting information flow between perception and action.

Future Steps. These limitations point to two directions: (1) a *time-causally ordered action tokenizer* that produces temporally ordered, compact token representations; and (2) a *unified discrete diffusion VLA* in which observation reasoning and action generation share the same backbone.

References

[1] J. Song, A. Vahdat, M. Mardani, and J. Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International conference on learning representations*, 2023.

[2] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han. Vlash: Real-time vlas via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.

[3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. pi0.5: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[5] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[6] K. Black, M. Y. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.

[7] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[8] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[9] A. Van Den Oord, O. Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[10] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.

[11] M. Matthews, M. Beukman, C. Lu, and J. Foerster. Kinetix: Investigating the training of general agents through open-ended physics-based control tasks. *arXiv preprint arXiv:2410.23208*, 2024.

[12] Y. Liu, J. I. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn. Bidirectional decoding: Improving action chunking via closed-loop resampling. *arXiv preprint arXiv:2408.17355*, 2024.

A Extended Related Works

B Simulation Experiments Details

Computation Recources.

Implementation Details. Check the code.

Training Recipe.

Hyperparameters.

C Real-world Experiments Details

Computation Recources.

Data Collection Pipeline.

Training Recipe.

Evaluation Setup and the Inference Latency Analysis.

Hyperparameters.